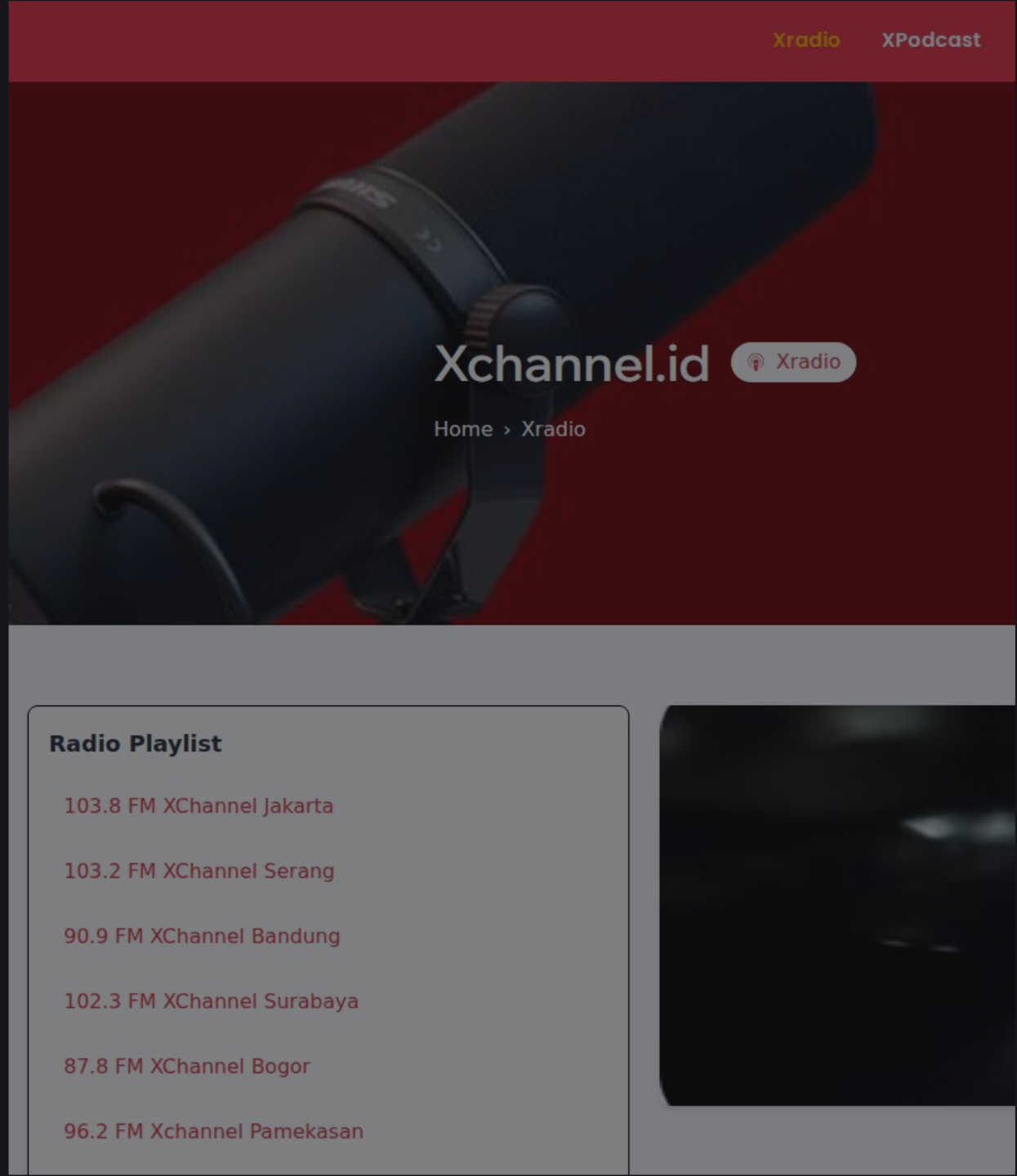
XRADIO · DASHBOARD

XChannel

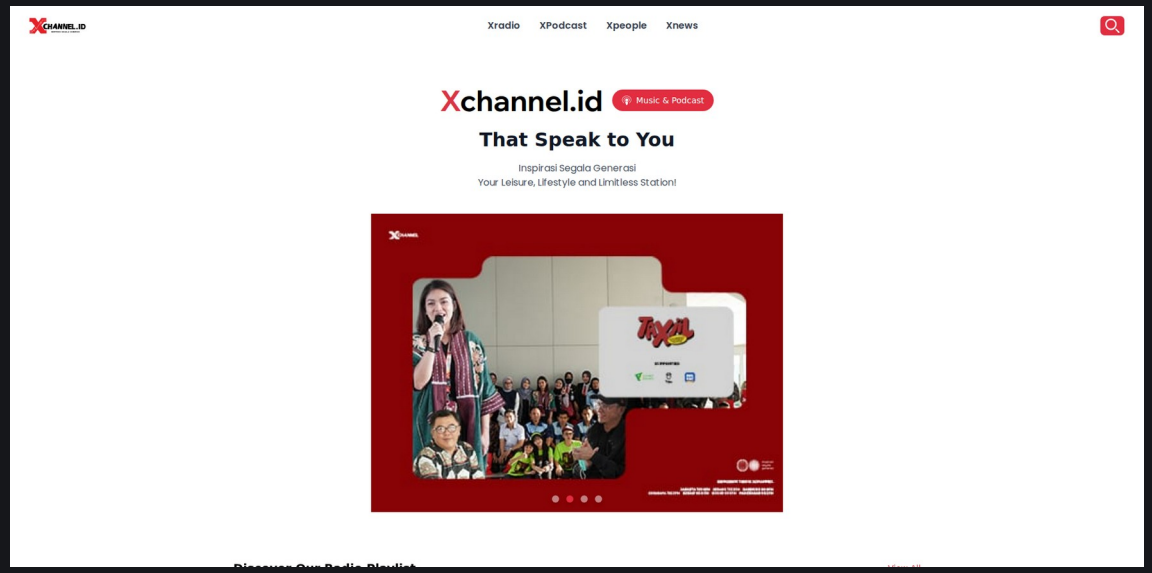
The broadcast schedule for local radio in several cities, and the system behind it

Enrico Saputra · Backend developer, Laravel dashboard

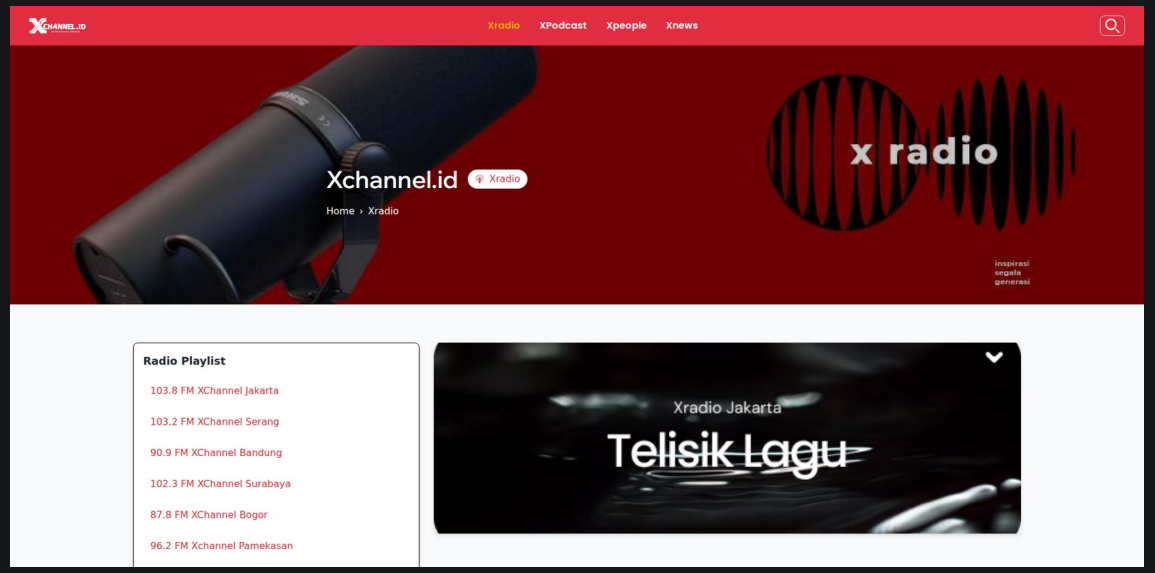
Client code and data are private. The demo rebuilds the radio part of the dashboard on the real data model, with made-up data.



xchannel.id, the side listeners see



Home: radio, podcasts, people and news



Xradio: stations per city and the show that is playing

The stations, shows, hosts and schedule on the Xradio page are managed in the Laravel dashboard. That is the part this demo rebuilds.

Screenshots of xchannel.id, October 2026

What I built at XChannel

The XChannel app plays each station's stream and shows what is on air. The broadcast team runs all of it from the dashboard.

What I built

- Stations: city, frequency, stream URL and status
- Shows and hosts
- Each station's weekly schedule, which the app reads
- Podcasts, chart, video, news: same dashboard, outside this demo

8
stations on xchannel.id today

7 → 1
per-city schedule tables become one

24/7
empty hours play automated music

About the app

XChannel is a media app, "That Speak to You": local radio in several cities (Xradio), podcasts (XPodcast), articles (Xpeople) and news (Xnews). Listeners play the radio streams live on the web or in the app.

Who uses it

Listeners

Play their city's station and see the show and host on air.

Broadcast team

Run the stations, shows, hosts and schedule in the dashboard.

Content team

Manage podcasts, articles and news in the same dashboard.

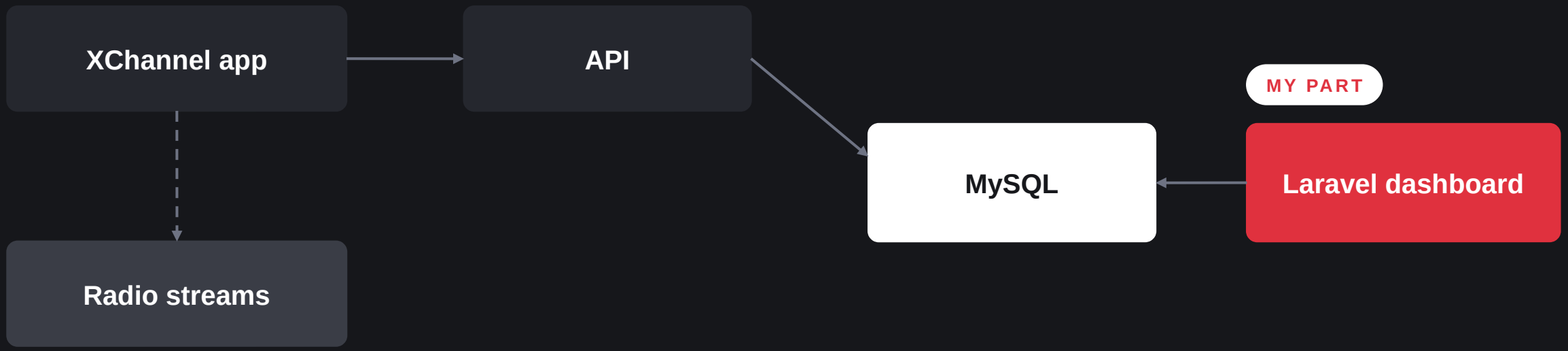
User journey

1. A listener opens Xradio and picks their city's station.
2. The station's stream starts playing.
3. The app shows the show and host on air right now.
4. The listener sees what is next and this week's schedule.
5. Behind the scenes, the broadcast team edits the schedule and the app follows.

What the dashboard contains

Menu	What it does	Data
Dashboard	On air and up next per station, empty hours, airtime per host and station	schedule_slots
Stations	City, frequency, stream URL and status, plus the station's week	stations
Schedule	Add, edit and remove slots; overlap check; one form for several days	schedule_slots
Shows	Title, description, hosts and active status	shows, host_show
Hosts	Name, bio, the shows they host and their airtime	hosts
Also in production (not in the demo)	Podcasts, music chart, video, news, games and member points	–

One database, two readers



The dashboard writes

Stations, shows, hosts and the schedule.

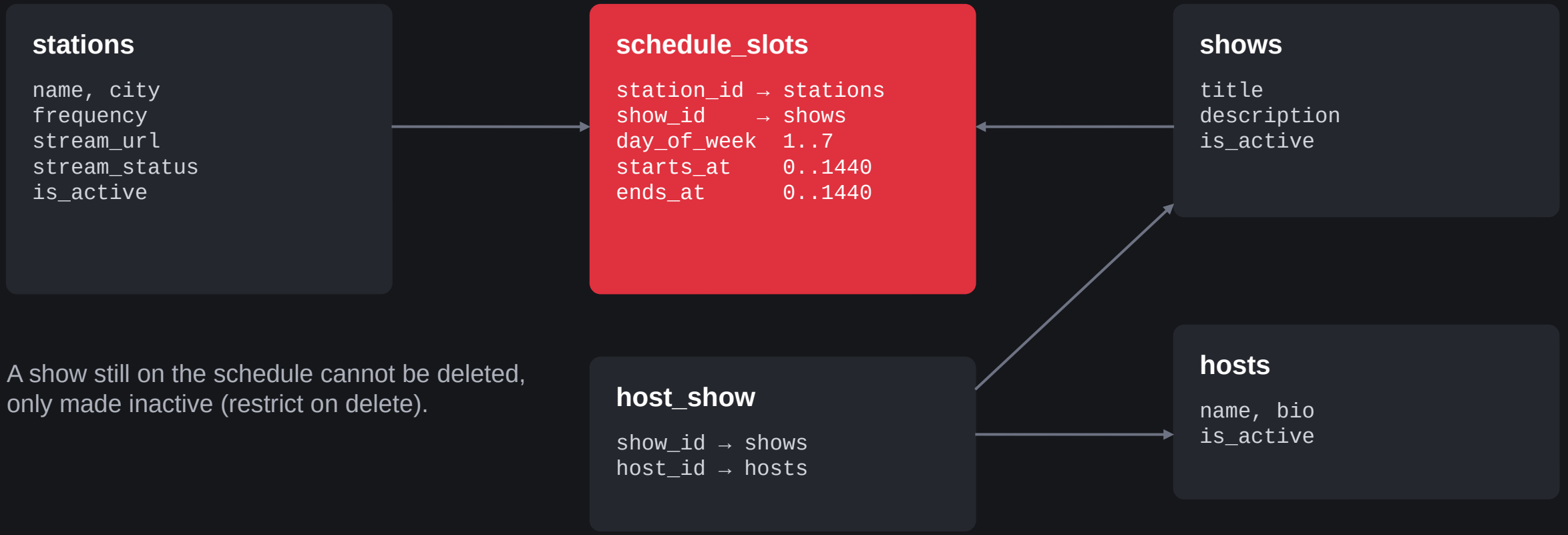
The app reads

"What is on air now?" through its API.

Rules on save

An overlapping schedule never reaches a listener.

Four tables and one pivot



A show still on the schedule cannot be deleted, only made inactive (restrict on delete).

Database structure (ERD)



`schedule_slots`

One row = one show, one day, one station.

`starts_at, ends_at`

Minutes since 00:00, from 0 to 1440.

`host_show`

Many-to-many between shows and hosts.

`Foreign key`

All restrict on delete: anything in use cannot be deleted.

From a table per city to one table

Production

7 TABLES

```
radio.api_jadwal = 'jadwal_radio_woman'

jdwl_radio_bandung
jdwl_radio_surabaya
jdwl_radio_suramadu  ...
  judul_siaran, hari_siaran,
  waktu_siaran, penyiar (text)
```

A new station means a new table. One name in api_jadwal does not match the real table.

Demo

1 TABLE

```
schedule_slots
  station_id  -> stations
  show_id     -> shows
  day_of_week, starts_at, ends_at

host_show (show_id, host_id)
```

A new station is one row. Foreign keys keep it consistent.

No-downtime migration: fill the new table from the old ones, keep a view per city for the old API, move readers one by one.

Time as numbers, not text

In production

`senin-jumat`	`Senin- Jumat`
`Monday-Friday`	`6:00 AM - 10:00 AM`
`08.00-09.00`	`20:00-24:00`

In the demo

450

minutes = 07:30

1440

minutes = 24:00

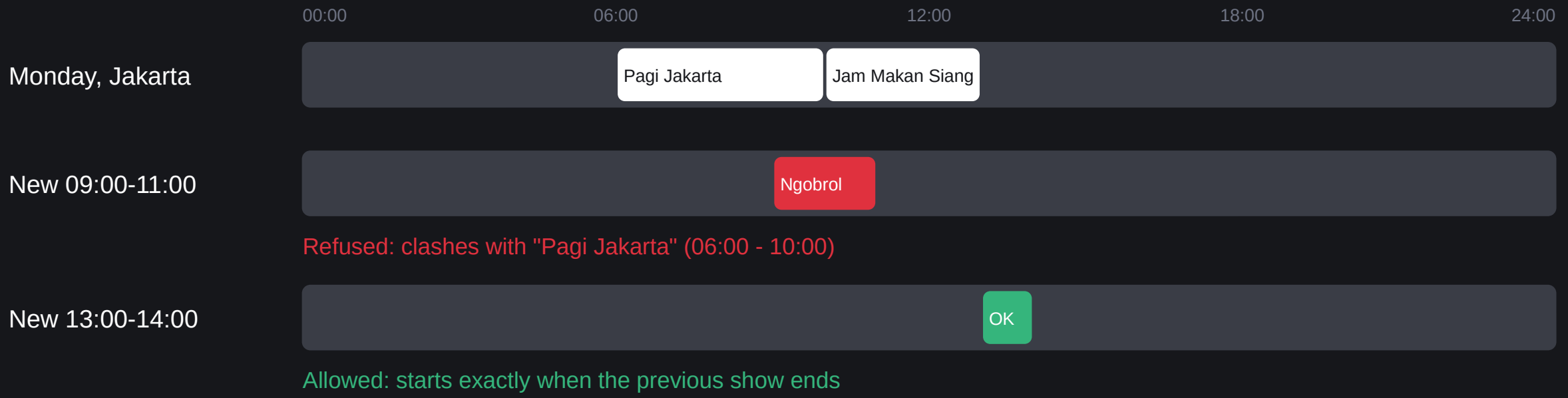
day_of_week 1-7 (ISO), one row per day. The form takes HH:MM only.

Two slots overlap when each starts before the other ends

```
starts_at < :end  AND  ends_at > :start
```

Touching (10:00 then 10:00) is fine.

The overlap rule on save



A clear message

Day, time and the show in the way.

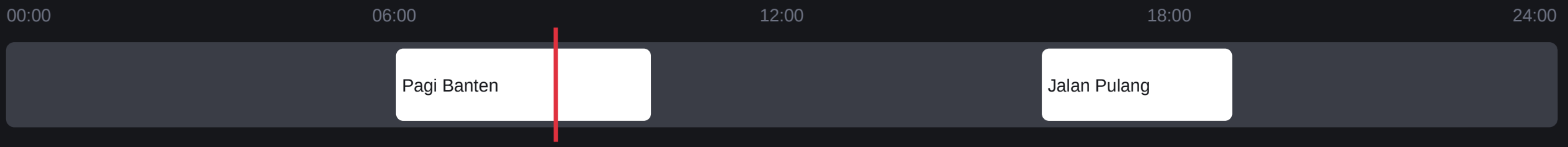
All days or none

Monday to Friday in one transaction.

Past midnight

Two slots: until 24:00, then from 00:00.

On air, up next, and empty hours



XChannel Banten, Monday 08:30. Red line = now; grey = automated music.

On air

Today's slot with start <= now < end. One indexed query.

Up next

The first slot after now, rolling into the next days for up to a week.

Empty hours

Worked out from the schedule, not stored. Airtime per station out of 168 hours.

Why this stack

Choice	Why
Laravel 10	Same as production; forms and validation are quick
MySQL	The production database. The same code runs on SQLite (tests) and PostgreSQL (online demo)
Blade + Tailwind	The timelines are plain HTML and CSS
PHPUnit	19 feature tests on the schedule rules

The overlap check is one query

```
public function clash(int $stationId, Day $day,
    int $start, int $end, ?int $ignoreId = null)
{
    // Two ranges overlap when each one starts
    // before the other ends. Touching is fine.
    return ScheduleSlot::with('show')
        ->where('station_id', $stationId)
        ->where('day_of_week', $day->value)
        ->where('starts_at', '<', $end)
        ->where('ends_at', '>', $start)
        ->when($ignoreId, fn ($q) =>
            $q->whereKeyNot($ignoreId))
        ->first();
}
```

Rules in one service

Clash, schedule, move, on air, up next, gaps.

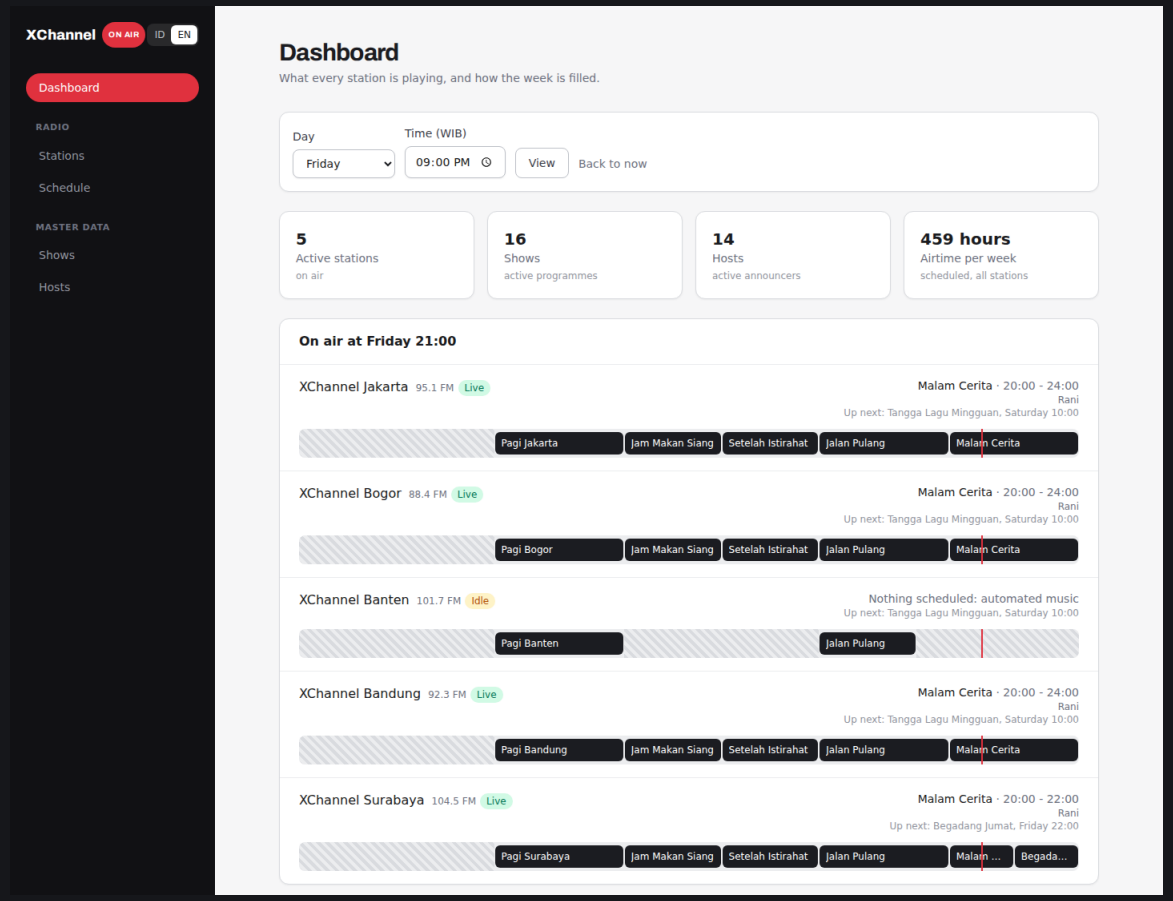
Enums, not numbers

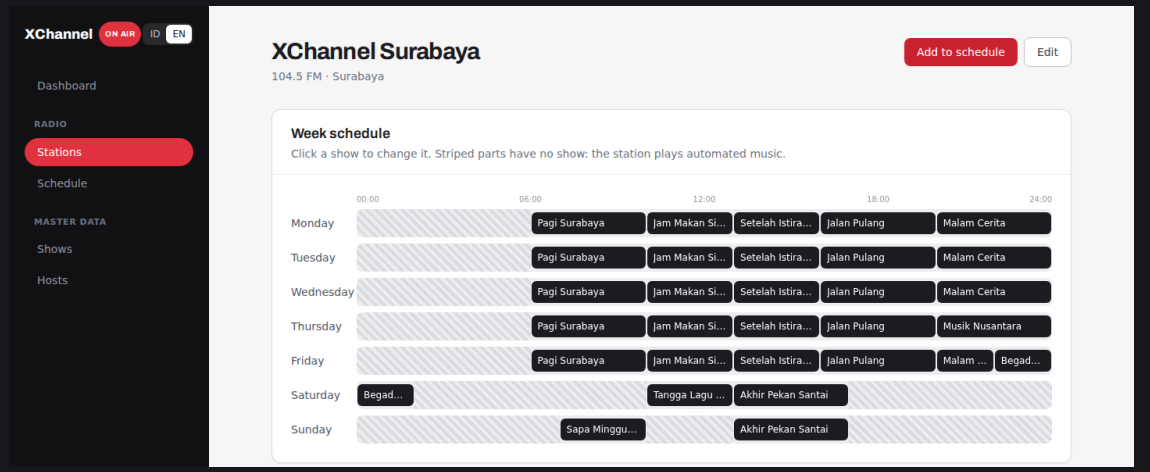
Day (ISO 1-7), StreamStatus.

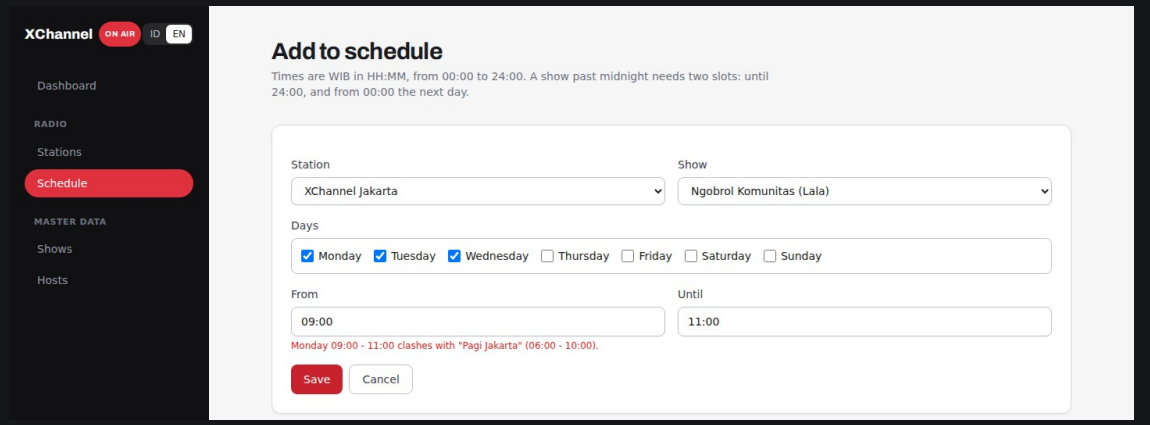
Thin controllers

FormRequests validate, Clock parses times.

Demo: dashboard, schedule, a refused slot







Dashboard: on air and airtime

Demo, slides and the real product

Live demo	https://xchannel-dashboard-demo.vercel.app
Slides (PDF)	https://xchannel-dashboard-demo.vercel.app/xchannel-walkthrough.pdf
Slides (PPTX)	https://xchannel-dashboard-demo.vercel.app/xchannel-walkthrough.pptx
Real product	https://xchannel.id/radio

The demo code is in a private GitHub repo; access is given during the interview. The dashboard speaks Bahasa Indonesia and English, and its data is restored every 3 hours.